Распределение регистров

© 2025 Юлия Кононова.
(редактировал: Косарев Дмитрий)

Creative Commons Attribution 4.0 International

1 Регистры

Регистры процессора — это небольшие области сверхбыстрой памяти внутри процессора, используемые для временного хранения данных, адресов и промежуточных результатов вычислений.

Регистры различаются в зависимости от архитектуры процессора:

- по назначению: целочисленные (GPR), для чисел с плавающей точкой (FPU), SIMD/векторные (MMX/SSE/AVX/NEON), специальные (счётчик стека, базовый указатель, счётчик команд и т.п.);
- по разрядности и взаимному наложению (aliasing): например, в x86-64 регистры AL, AX, EAX, RAX ссылаются на один и тот же физический регистр разной ширины, что наглядно показано на Рис. 1.
- по доступности: некоторые регистры зарезервированы (например, RSP под стек в x86-64), часть — должна сохраняться по соглашению о вызовах (callee-saved), а часть может быть «испорчена» вызовом (caller-saved);

- по архитектурным ограничениям инструкций: отдельные инструкции требуют конкретные регистры (например, `MUL/IMUL/IDIV` в x86 используют RAX/RDX), либо регистровые пары/выравнивание.

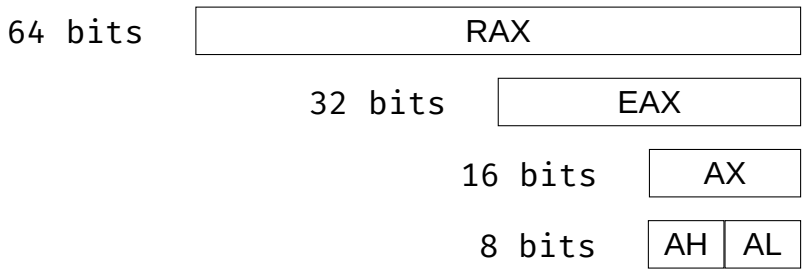


Рис. 1: Пример иерархии и наложения (aliasing) регистров общего назначения в архитектуре x86-64. Регистры младших разрядностей (EAX, AX, AH, AL) являются физически частью старшего 64-битного регистра RAX.

У нас есть память имеющая быстрый доступ (доступ к регистрам гораздо быстрее, чем к оперативной памяти), но есть проблема — ее мало и хочется максимально эффективно использовать это ограниченное пространство, чтобы сократить операции загрузки и выгрузки данных из памяти, тем самым ускорив работу программы.

2 Распределение регистров

Распределение регистров — это процесс выбора, какие переменные и промежуточные значения программы будут храниться в регистрах процессора, а какие — в оперативной памяти.

В промежуточном представлении обычно неограниченное количество виртуальных регистров. А у конкретного процессора их конкрет-

ное число. Из-за этого возникает **задача распределить виртуальные регистры по физическим**.

Для решения этой задачи используются различные алгоритмы. В данном конспекте рассматриваются алгоритмы **LinScan** и **Puzzle solving**.

2.1 Основные понятия

Перед тем как переходить к алгоритмам, необходимо ввести несколько основных определений.

Переменная считается **живой** в некоторой точке программы, если:

- она уже была определена (ей присвоено значение);
- и существует путь исполнения, при котором значение из нее может быть прочитано (использовано) до следующего присваивания.

Промежуток активности переменной (live range) — это множество точек в программе, где переменная живая. Важно отметить, что live range может быть несвязным, то есть содержать ”дыры”, когда переменная временно не используется.

Интервал активности переменной (live interval) — это непрерывный отрезок от первого определения переменной до её последнего использования в программе. Интервал активности является упрощением промежутка активности. Даже если в середине этого отрезка есть ”дыры ¨где переменная неживая, интервал всё равно считается непрерывным.

Наглядно разницу между промежутком и интервалом активности демонстрирует Рис. 2.

Spilling — процесс выгрузки содержимого переменной из регистра в память, когда не хватает доступных регистров для хранения всех активных переменных.

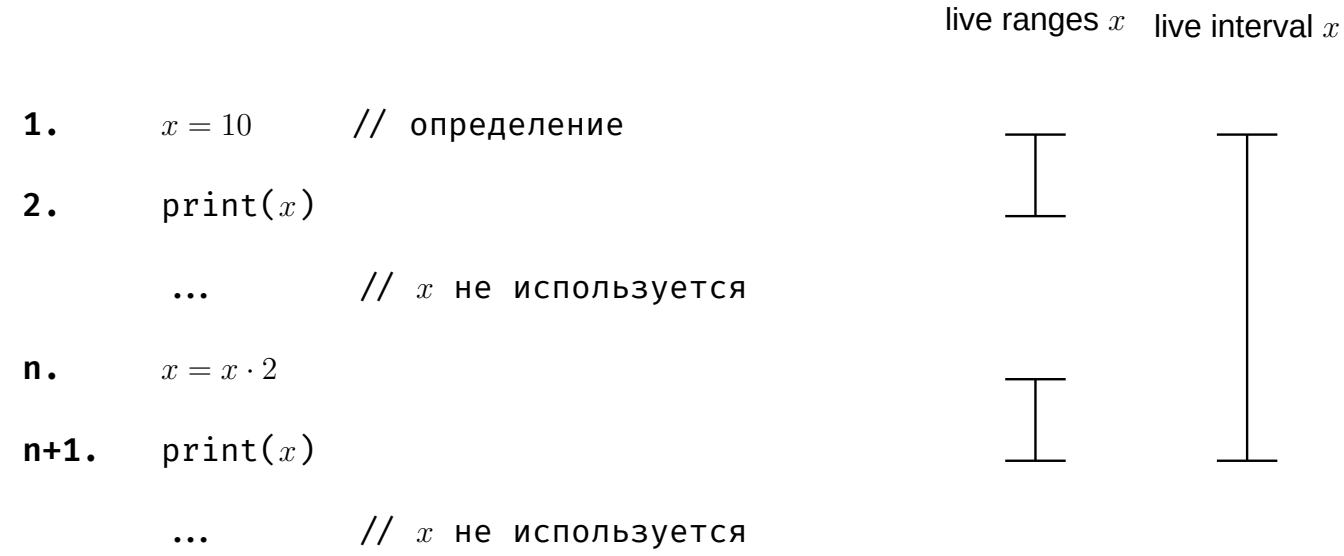


Рис. 2: Сравнение live ranges и live interval для переменной x. Промежуток активности состоит из двух несвязных сегментов, в то время как интервал активности покрывает весь диапазон от первого до последнего использования одной непрерывной областью, игнорируя ”дыры”.

2.2 LinScan

Хочется получать ответ более-менее быстро (например, для JIT-компиляции). Для этого часто используются линейные алгоритмы. Один из них — **LinScan**.

Суть алгоритма заключается в линейном проходе по списку интервалов активности переменных и попытке назначить каждой переменной регистр таким образом, чтобы минимизировать количество обращений к памяти, т.е. уменьшить количество операций spilling.

2.2.1 Алгоритм

Перед работой алгоритма для каждой переменной вычисляется её интервал активности (этот этап может быть выполнен за один проход по графу потока управления). Эти интервалы сортируются по времени (номеру инструкции) начала их жизни.

Алгоритм обходит отсортированные интервалы один за другим и выполняет следующее:

1. Перед обработкой нового интервала освобождаются все регистры, занятые интервалами, которые уже завершились;

2. Назначается регистр:

    - Если доступен свободный физический регистр, он назначается текущему интервалу;
    - Если свободных регистров нет, вызывается процедура вытеснения (spilling) — интервал с наименьшим приоритетом (например, с наибольшим временем жизни или меньшей частотой использования) сохраняется в памяти, чтобы освободить регистр;

Рассмотрим несколько примеров (примеры взяты из Стэнфордского курса [3]).

2.2.2 Примеры

Пример 1

Ниже представлена последовательность шагов алгоритма LinScan для программы с семью переменными (a-g) и четырьмя доступными регистрами (R0-R3).

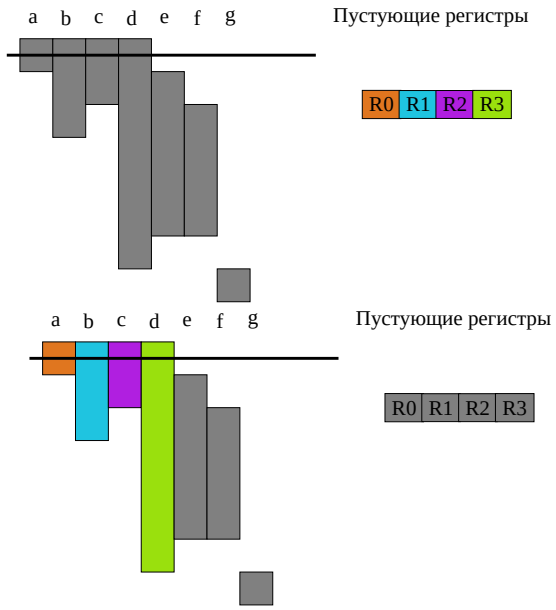


Рис. 3: Алгоритм начинает обход интервалов активности. Первые переменные получают свободные регистры.

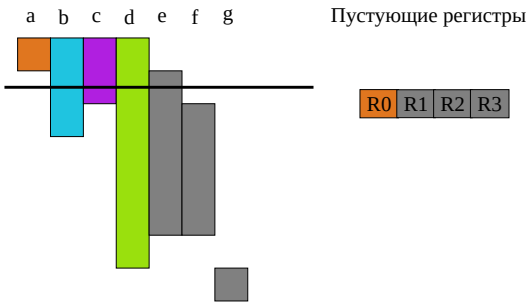


Рис. 4: Переменная a завершает свой интервал активности, регистр R0 освобождается для следующих переменных.

Пример 2 — с необходимостью вытеснения

Данный пример демонстрирует ситуацию, когда количество активных переменных превышает число доступных физических регистров, что приводит к необходимости **вытеснения (spilling)**.

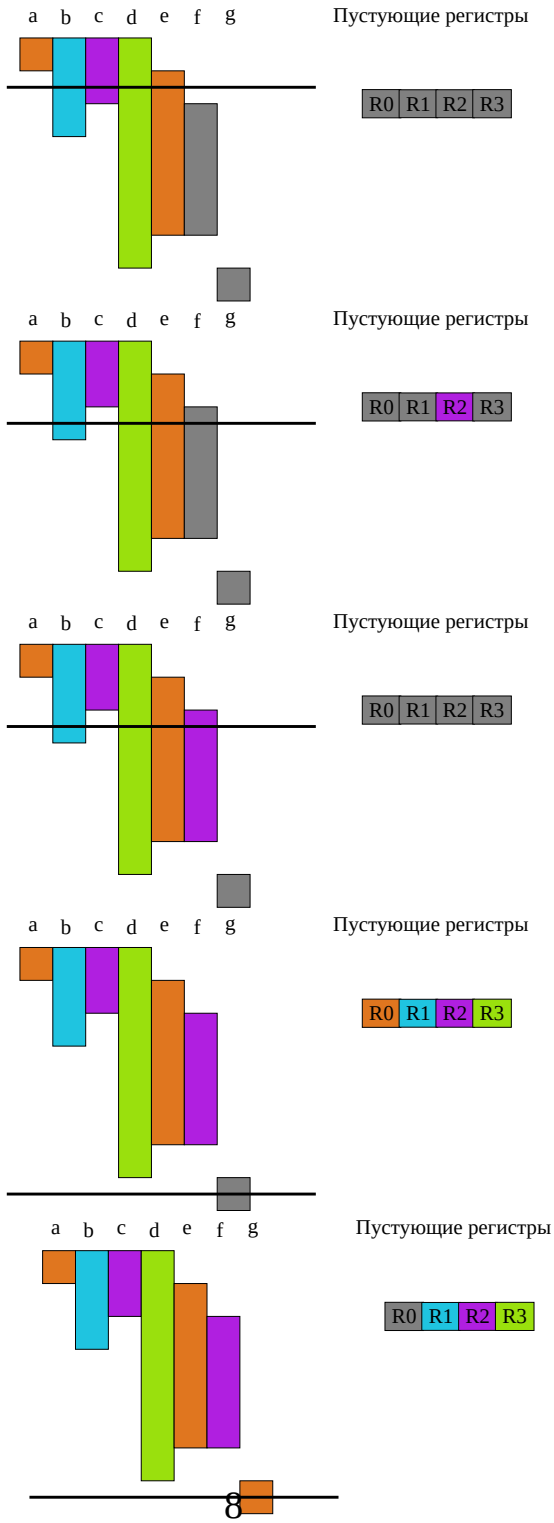


Рис. 5: Новые переменные занимают освободившиеся регистры. Алгоритм продолжает распределение по мере освобождения ресурсов.

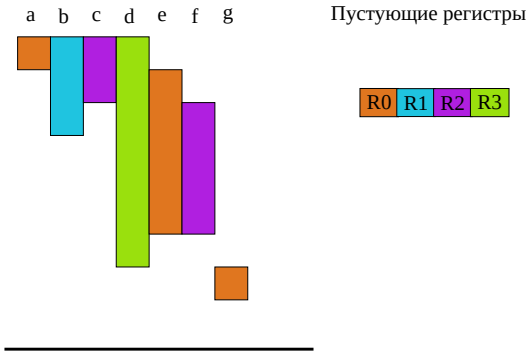


Рис. 6: Все переменные обработаны. Алгоритм завершил распределение регистров.

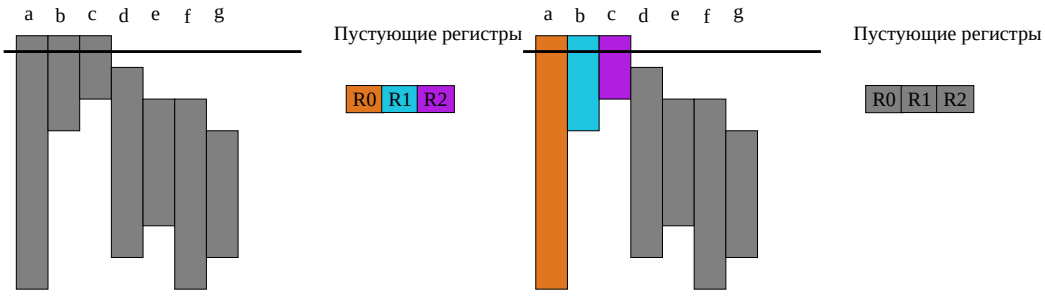


Рис. 7: Аналогично первому примеру алгоритм начинает обход интервалов активности.

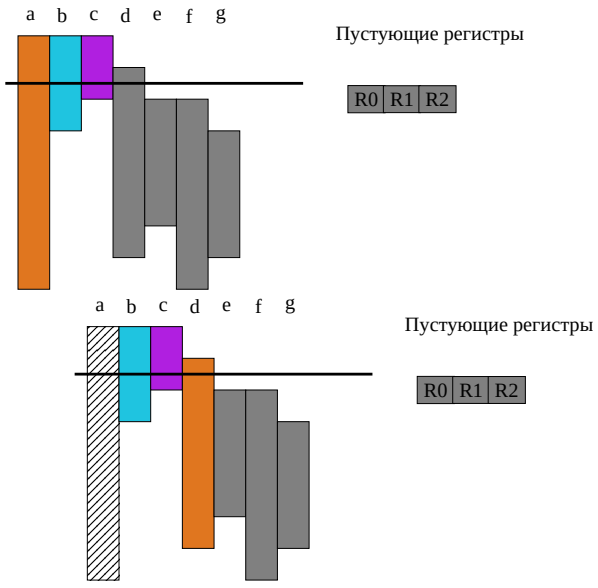


Рис. 8: Все регистры заняты, но появляется переменная **d**, которой требуется регистр для работы. Алгоритм выбирает для вытеснения (spilling) переменную **a**. Значение переменной **a** сохраняется в память. Переменная **d** получает освободившийся регистр R0.

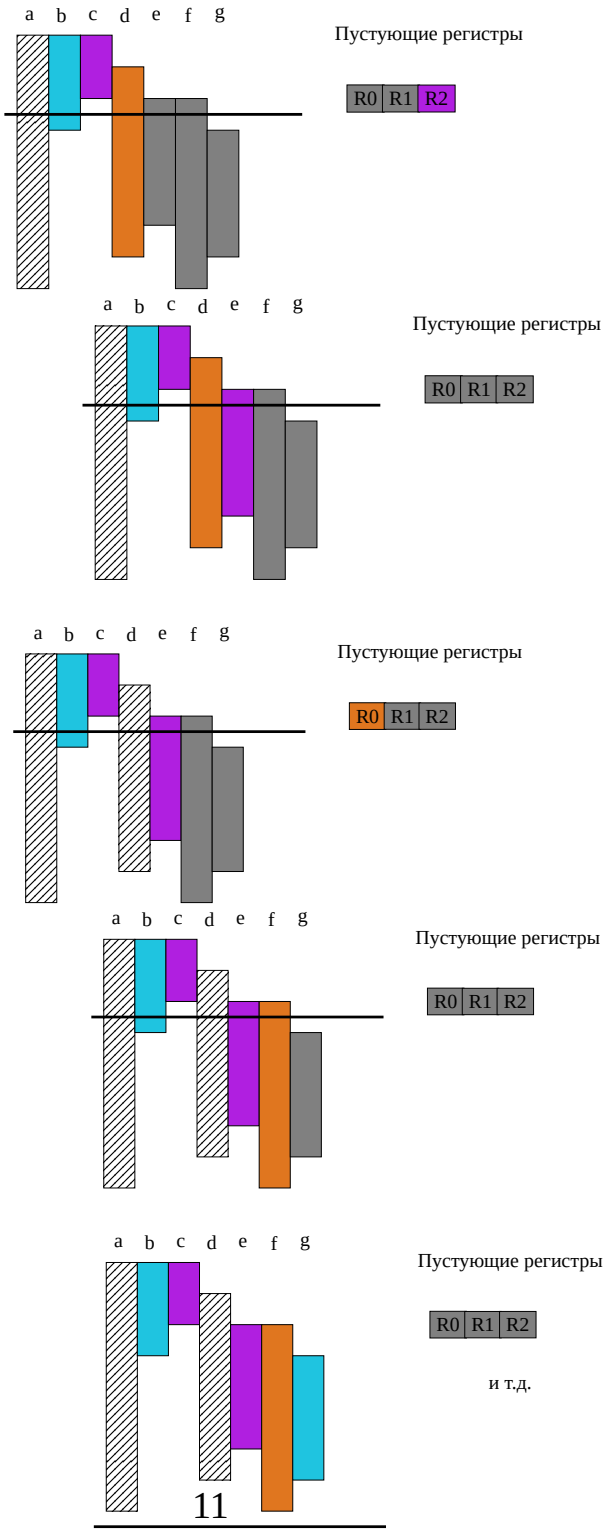


Рис. 9: Продолжение процесса распределения с учетом механизма spilling. Алгоритм продолжает назначать регистры переменным, при необходимости освобождая их через вытеснение ранее размещенных переменных. Процесс повторяется до полного распределения всех пе-

Пример 3

Для повышения качества распределения вместо непрерывных интервалов можно **использовать более точные промежутки активности (live ranges)**. Такой подход учитывает «дыры» — неактивные участки внутри общего времени жизни переменной, что в ситуациях высокого регистрового давления («узких местах») позволяет сократить количество spill‘ов.

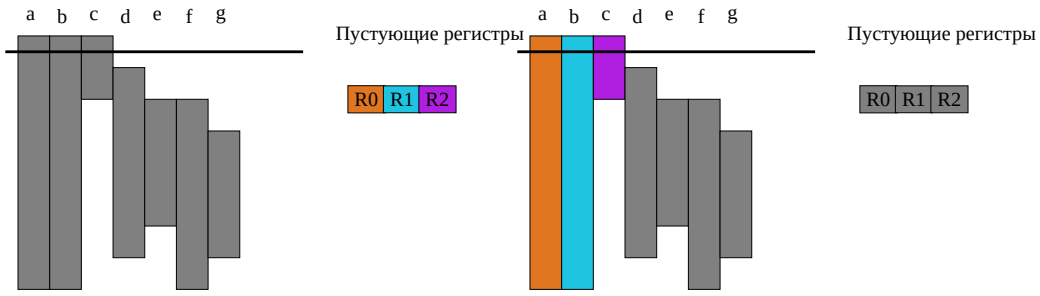


Рис. 10: Аналогично первому примеру алгоритм начинает обход интервалов активности.

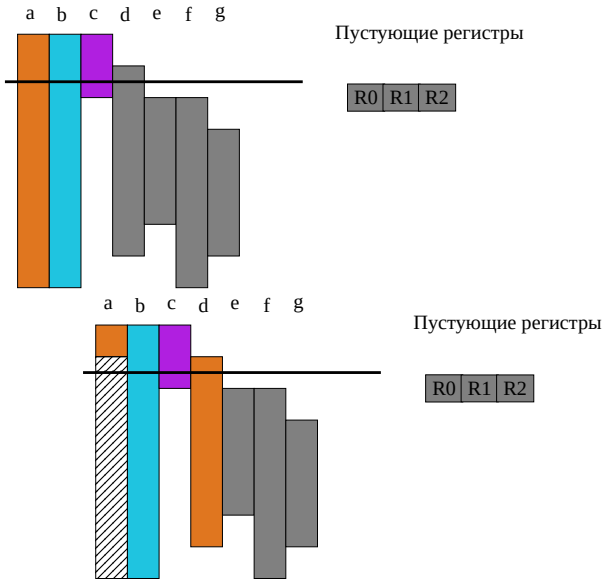


Рис. 11: Возникает ситуация как на Рис. 8, но, в отличие от нее, теперь мы оперируем **промежутками активности**. Первый промежуток активности переменной **a** завершился, и сейчас эта переменная не используется. Поэтому, мы можем ее вытеснить и положить в освободившийся регистр R0 переменную **d**.

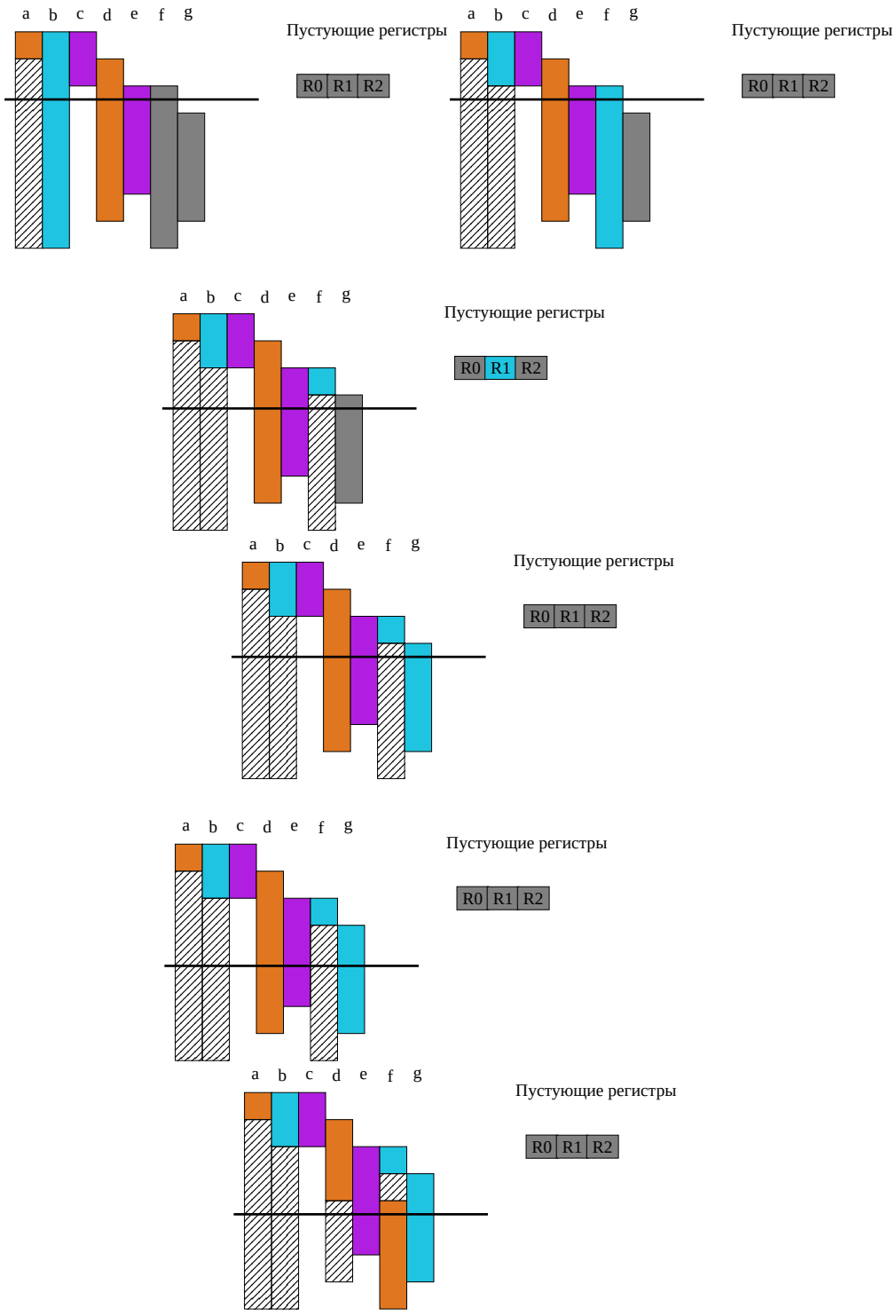


Рис. 12: Продолжение процесса распределения регистров. Алгоритм продолжает отслеживать завершение и начало промежутков активности переменных, динамически перераспределяя регистры между переменными. Такой подход снижает число операций с памятью за счет более точного учета реального времени жизни переменных. Процесс

2.2.3 Плюсы и минусы

Преимущества:

- Очень эффективный (после вычисления интервалов активности работает за линейное время);
- Простота реализации;
- Предсказуемость поведения;
- Один проход, можно порождать код по мере интерпретации;
- Часто используется при JIT-компиляции.

Недостатки:

- Неточен, т.к. используются интервалы вместо промежутков активности;
- Меньшее качество распределения — по сравнению, например, с графовыми методами, LinScan может давать большее количество spill’ов;
- Отсутствие глобального анализа — не учитывает конфликты регистров на больших участках кода, что может снизить производительность.

2.3 Puzzle solving

По сути, задача распределения регистров — это задача сопоставления переменных и регистров. Основное ограничение — это интервалы активности переменных.

Например, рассмотрим Рис. 13. На нем представлена программа без ветвлений — прямолинейный код. Интервалы активности переменных можно изобразить цветными полосами. Справа — регистры.

Допустим, мы работаем с архитектурой, где регистр может быть разделён на две половины (например, $X = XH + XL$). Некоторые переменные ”толстые” (например, B и E) — им нужен полный регистр, другие ”тонкие” — им достаточно половины.

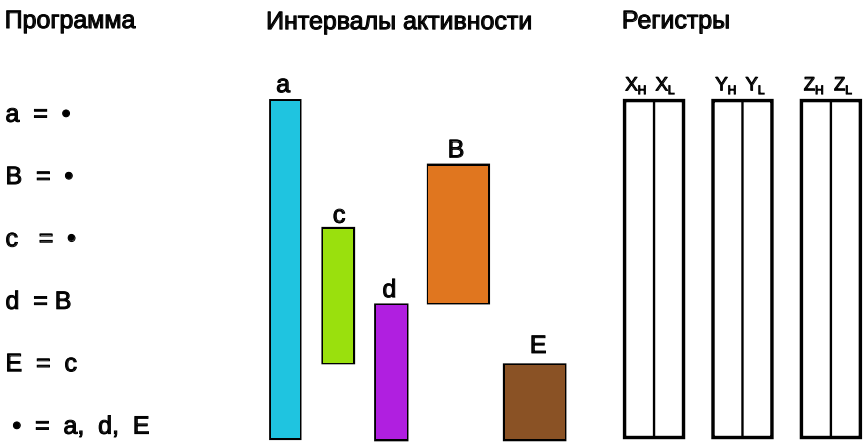


Рис. 13: Пример задачи распределения регистров.

Распределение переменных по регистрами — это задача упаковки полос в ограниченное число регистров. На Рис. 14 показано одно из возможных распределений для примера выше.

Это решение оптимально: для данной программы нельзя использовать меньше трёх регистров.

Однако, если внимательно посмотреть на конец программы (Рис. 15), можно заметить проблему: переменные a и d оказываются в разных регистрах, хотя могли бы поместиться в один. Из-за этого переменная E вынуждена размещаться в третьем регистре.

Чтобы упростить задачу, предлагается разбивать интервалы активности переменных на более короткие. Если интервалы короче, их проще ”упаковать” в регистры.

Для этого используется специальное представление программы — элементарная форма, где каждый жизненный интервал очень короткий

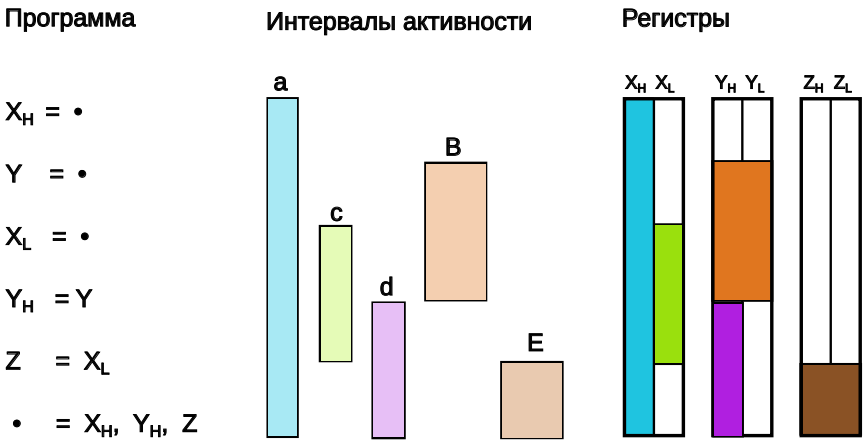


Рис. 14: Пример распределения переменных по регистрам для программы, представленной на Рис. 13.

(максимум две точки программы). На Рис. 16 видно, что теперь жизненные интервалы стали очень короткими.

В результате, как видно на Рис. 17, для той же программы теперь достаточно двух регистров вместо трёх!

Обратите внимание, что переменная a была перемещена из регистра XH в регистр YL перед последней инструкцией. Это возможно благодаря вставке дополнительной инструкции копирования (на Рис. 17 она отмечена серым цветом).

В итоге, задача распределения регистров формулируется как решение набора головоломок (puzzles):

- Для каждой инструкции программы формируется отдельная головоломка. На Рис. 18 представлена одна из головоломок.
- Игровое поле головоломки задаётся множеством регистров. В примере на Рис. 18 игровое поле состоит из трёх областей, каждая из которых разделена на два столбца и две строки.
- Детали головоломки задаются переменными программы. Например, выделенная головоломка состоит из четырёх частей.

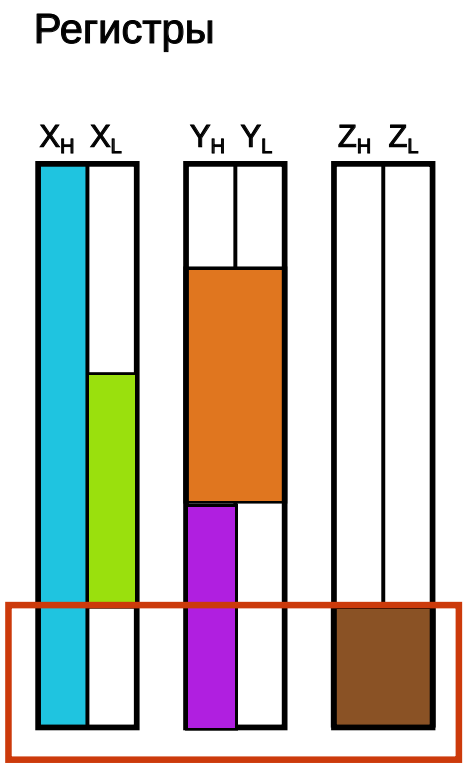


Рис. 15

- Есть определённые правила размещения частей. Оранжевый элемент, представляющий переменную B, может быть помещен только в верхнюю строку области. Однако мы можем свободно выбирать, в какую именно область. Маленький фиолетовый элемент, представляющий переменную d, может быть вставлен только в нижнюю строку. Зелёный и голубой элементы охватывают обе строки. Однако они занимают только один столбец.

Главное преимущество: такие головоломки можно решать оптимально за полиномиальное время, даже если часть поля уже занята. Любую программу можно привести к элементарной форме, и она не потребует больше регистров, чем исходная.

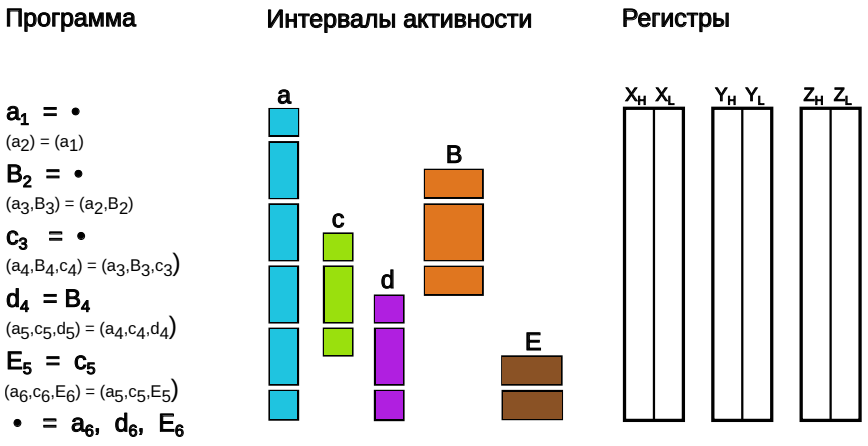


Рис. 16

2.3.1 Состав игры

Головоломка состоит из:

- Игрового поля — это наше множество физических регистров.
- Набора деталей (пазлов) — это переменные программы, которые нужно разместить.
- Правила игры — это ограничения, которые нельзя нарушать (например, наложения жизненных интервалов, архитектурные требования, зарезервированные регистры).

Детали не могут перекрываться на игровом поле, и часть деталей уже размещена на нём (например, могут существовать зарезервированные регистры). Задача состоит в том, чтобы разместить оставшиеся детали на игровом поле.

2.3.2 Виды игрового поля и пазлов

Каждое игровое поле состоит из нескольких отдельных областей, каждая из которых разделена на два ряда. Авторы игры выделяют три

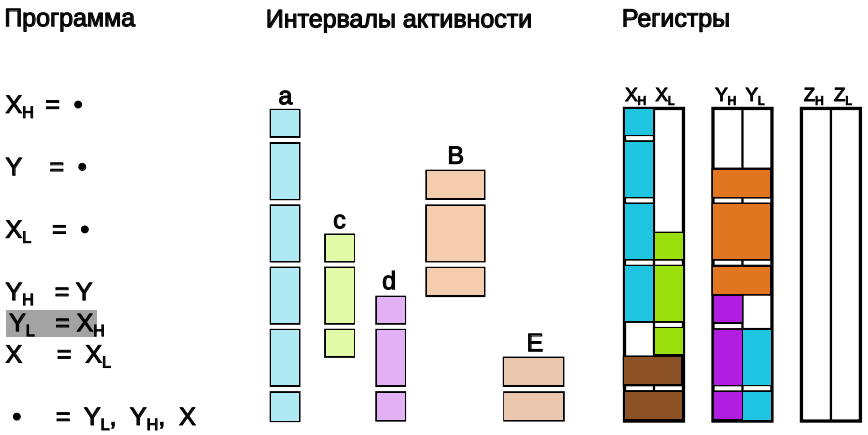


Рис. 17

основных типа головоломки (Рис. 19), которые определяются её полем и формой деталей:

- **Тип 0.** Каждая область игрового поля состоит только из одного столбца. Для заполнения доступны всего три типа деталей, показанных на Рис. 19:

  – Деталь X: размещается в любой ячейке верхнего ряда.

  – Деталь Z: размещается в любой ячейке нижнего ряда.

  – Деталь Y: занимает весь столбец целиком (обе ячейки).

  Решение головоломки типа 0 тривиально и выполняется за линейное время от количества областей: сначала размещаются все детали типа Y, а затем — оставшиеся детали X и Z.

- **Тип 1.** Головоломки Типа 1 имеют области с двумя столбцами и шесть различных типов деталей (Рис. 19).

  Правила размещения аналогичны типу 0: детали X занимают верхний ряд, Z —нижний, а Y — оба ряда. Ключевое ограничение:

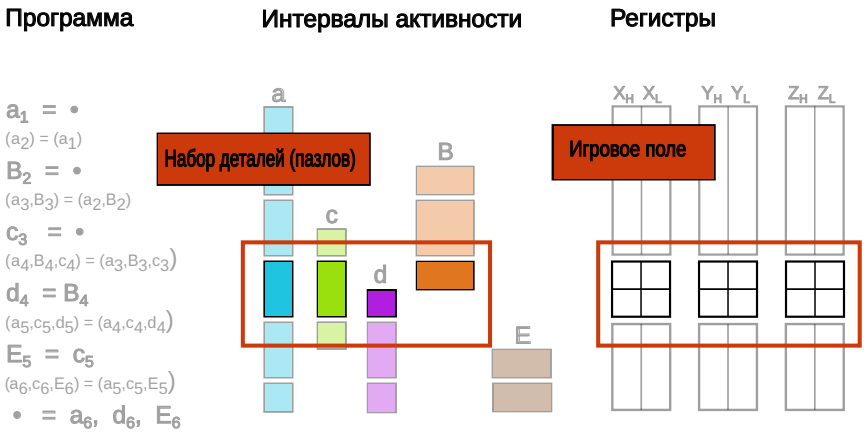


Рис. 18

деталь, занимающая два столбца, должна полностью находиться в пределах одной области. Далее будет представлен алгоритм для решения головоломок этого типа за линейное время.

- **Тип 2.** Здесь каждая область состоит уже из четырех столбцов, а количество типов деталей возрастает до девяти (Рис. 19). Правила размещения аналогичны предыдущим типам, но с одним важным дополнением: детали шириной в два столбца должны размещаться либо в первой (столбцы 1-2), либо во второй (столбцы 3-4) половине области, не пересекая её середину.

  Решение головоломок второго типа остается открытой проблемой.

Определяет тип головоломки множество регистров в целевой архитектуре. Возможность группировки регистров в пары или четвёрки определяет количество столбцов на игровом поле.

Два регистра образуют псевдоним (aliasing), если присвоение значения одному из них может изменить значение другого.

- **Тип 0**: Встречается в архитектурах, где регистры не объединяются. Например, регистры в PowerPC (R0-R31) или целочисленные

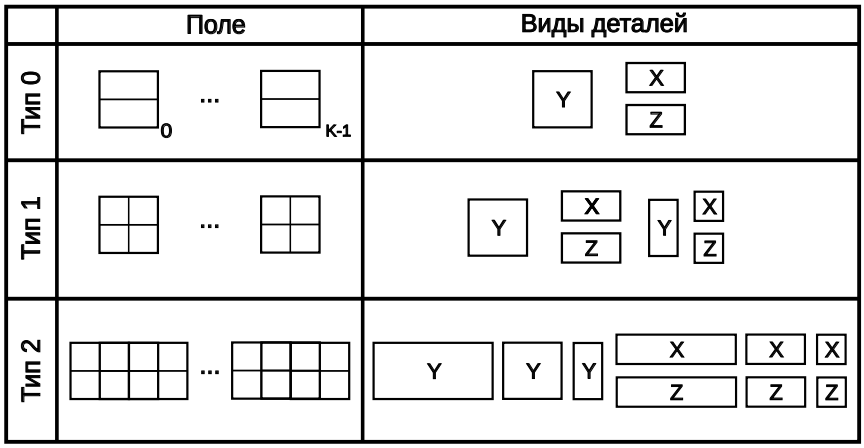


Рис. 19: Три вида головоломки.

регистры ARM. Каждый регистр имеет уникальное имя, поэтому игровое поле состоит из областей с одним столбцом.

- **Тип 1**: Характерен для архитектур, где два регистра меньшей разрядности могут объединяться в один регистр большей разрядности. Классический пример — регистры с плавающей запятой в ARM, где два 32-битных регистра одинарной точности (например, S0 и S1) образуют один 64-битный регистр двойной точности (D0). Такая пара формирует область с двумя столбцами.

- **Тип 2**: Встречается реже. Примером служат регистры с плавающей запятой в SPARC V8, где поддерживается несколько уровней объединения: два 32-битных регистра формируют 64-битный, а два 64-битных — один 128-битный. Это приводит к созданию областей с четырьмя столбцами.

2.3.3 Гибридный тип

Некоторые архитектуры представляют собой комбинацию нескольких типов головоломок. Каноническим примером является регистро-

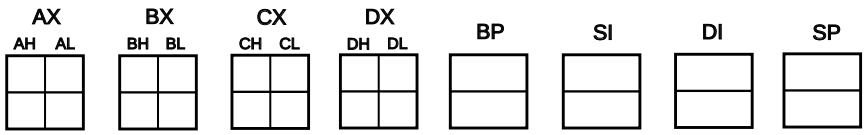


Рис. 20: Игровое поле для регистров x86.

вый файл 32-битной архитектуры x86, который представляет собой гибрид типов 0 и 1 (Рис. 20).

Часть типа 1 включает регистры AX, BX, CX и DX. Каждый из этих 16-битных регистров является псевдонимом для пары 8-битных регистров (например, AX для AL (младшая часть) и AH (старшая часть)). Их 32-битные расширения (EAX, EBX и т.д.) также являются псевдонимами для этих же пар. Поэтому в рамках головоломки EAX и AX рассматриваются как единая структура, поскольку они управляют одной и той же парой AL и AH.

Часть типа 0 состоит из регистров BP, SI, DI и SP. Эти регистры не имеют разделяемых на пары псевдонимов и потому формируют области с одним столбцом.

Для решения такой головоломки задача разбивается на части: сначала решается часть, соответствующая типу 0, а затем — часть, соответствующая типу 1.

2.3.4 Подготовка к игре

Перед началом игры происходит:

1. преобразование исходной программы в элементарную форму;
2. сопоставление элементарной формы с деталями головоломки.

2.3.5 Элементарная форма

Прежде чем приступить к решению головоломок, исходную программу необходимо преобразовать в **элементарную форму**. Это спе-

циальное представление программы, являющееся подмножеством SSA-формы (Static Single Assignment), в котором диапазоны жизни переменных содержат не более двух программных точек.

Идея: разбить глобальную задачу распределения регистров на последовательность локальных головоломок.

В результате преобразования каждая переменная **активна не более чем в двух последовательных точках программы**.

Преобразование любой программы в элементарную форму выполняется за три шага:

1. **Преобразование в SSA-форму.**

   На этом шаге все переменные переименовываются для обеспечения уникальности каждого присваивания. В точки слияния потоков управления (например, в начале циклов или после условных операторов) вставляются φ-функции для объединения различных версий одной и той же переменной.

2. **Преобразование в SSI-форму (Static Single Information).**

   Этот шаг является обратным к предыдущему. В точки ветвления потока управления (например, в конце базовых блоков) добавляются π-функции. Они являются двойственными по отношению к φ-функциям: если φ-функции объединяют потоки значений, то π-функции, наоборот, разделяют их, создавая новые версии переменных для каждой из ветвей.

3. **Вставка параллельных копий.**

   Наконец, между каждой парой последовательных инструкций внутри базового блока вставляются операции параллельного копирования с последующим переименованием переменных, активных в этой точке.

Весь процесс преобразования может быть выполнен за полиномиальное время.

Рассмотрим полный пример на Рис. 21. Слева показана исходная программа с четырьмя базовыми блоками. Справа — та же программа, преобразованная в элементарную форму.

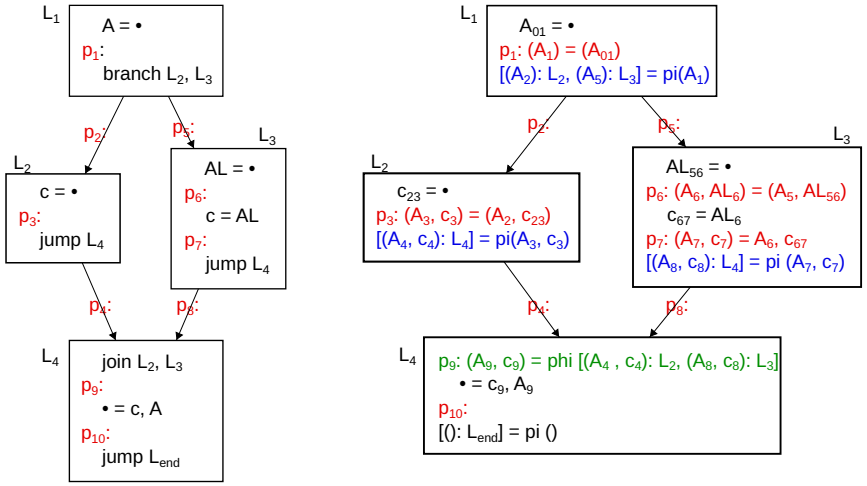


Рис. 21

Сначала, чтобы преобразовать программу в форму SSA, надо переименовать переменные, которые определены более одного раза, и вставить φ-функцию (обозначена зеленым), которая объединяет значения переменной, приходящие из разных блоков.

Затем для перехода к SSI-форме были добавлены π-функции (синие), которые разделяют диапазоны жизни переменных на выходах из блоков.

Наконец, между всеми последовательными инструкциями были вставлены параллельные копии (красные) и выполнено финальное переименование.

После этих шагов диапазон жизни каждой переменной стал настолько коротким, что для каждой инструкции можно сформулировать и решить отдельную головоломку распределения регистров.

2.3.6 От элементарной программы к деталям головоломки

Для каждой инструкции создается своя головоломка, где «детали» — это переменные, активные в этой точке. Правила преобразования переменных в детали очень просты:

- Деталь X: Если диапазон жизни переменной заканчивается на текущей инструкции (она используется, но её значение дальше не нужно), она становится деталью X.
- Деталь Z: Если диапазон жизни переменной начинается на текущей инструкции (она определяется здесь), она становится деталью Z.
- Деталь Y: Если диапазон жизни переменной проходит сквозь инструкцию (она была активна до и остается активной после), она становится деталью Y.

Почему игровое поле имеет два ряда? Эта структура моделирует повторное использование регистров. Регистр можно выделить под одну переменную, жизнь которой заканчивается (деталь X в верхнем ряду), и сразу же после этого — под новую переменную, жизнь которой начинается (деталь Z в нижнем ряду). Деталь Y занимает обе ячейки, показывая, что регистр занят на протяжении всей инструкции и не может быть использован для других целей.

Размер детали определяется типом соответствующей переменной. Например, в архитектуре x86 8-битная переменная (требующая регистр AL) станет деталью шириной 1, а 16-битная (AX) — деталью шириной 2.

На Рис. 22 слева представлена программа в простейшем виде, а справа — головоломки, созданные для этой программы. В этом примере архитектуры всего два регистра, которые разделены на пары. Множество регистров показан в верхней части рисунка. Вы можете заметить, что в двух головоломках некоторые области уже заполнены. Это

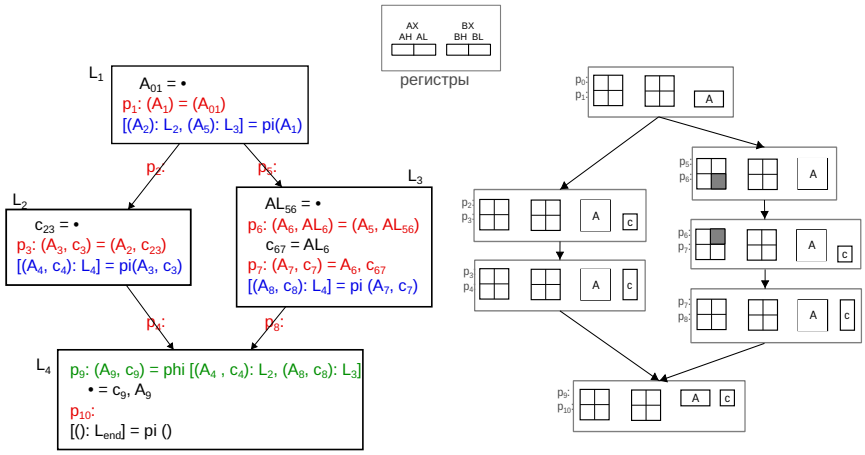


Рис. 22

связано с предопределенными регистрами. Некоторые соглашения в целевой архитектуре требуют хранения некоторых переменных в определённых регистрах. Примером может служить инструкция деления в X86. Для выполнения команды Div требуется, чтобы делимое хранилось в регистре AX. Остаток хранится в AH, а частное — в AL.

2.3.7 Правила

Решение головоломок типа 1 основывается на том, что каждая область состоит всего из четырех ячеек. На Рис. 23 перечислены все возможные способы заполнения области. Данный набор комбинаций образует визуальную грамматику, которая служит языком для программирования решателей головоломок первого типа.

- Программа — последовательность операторо.
- Оператор — либо правило «r», либо условный оператор.
- Правило — это шаблон. Шаблоны определяются предварительной раскраской области (серые ячейки).

(Программа) p := s_1 ... s_n

(Оператор) s := r | r:s

(Правило) r :=

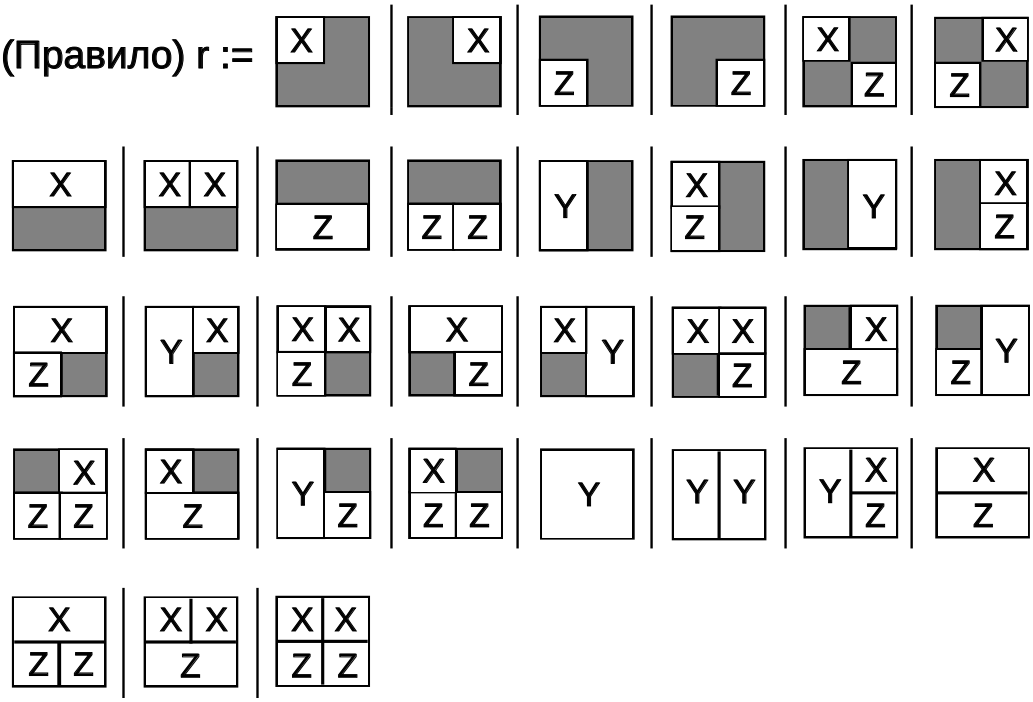


Рис. 23

Движок для решения головоломок решает одну область за раз, заполняя её элементами. Для работы программы площадь деталей должна быть равна площади пустых ячеек. Если площадь деталей больше, то головоломка не будет решена. Если площадь деталей меньше, в головоломку добавляется несколько деталей типа X и Z размера 1, пока не получается та же площадь.

Правило объясняет, как заполнить область. Оно работает по принципу сопоставления с образцом.

Механизм решения головоломок типа 1 продемонстрирован на Рис. 24. Решатель выполняет предопределенную программу, состоящую из по-

следовательности операторов. В данном примере программа содержит два оператора, второй из которых является условным: он предлагает несколько альтернативных правил для одного и того же шаблона области.

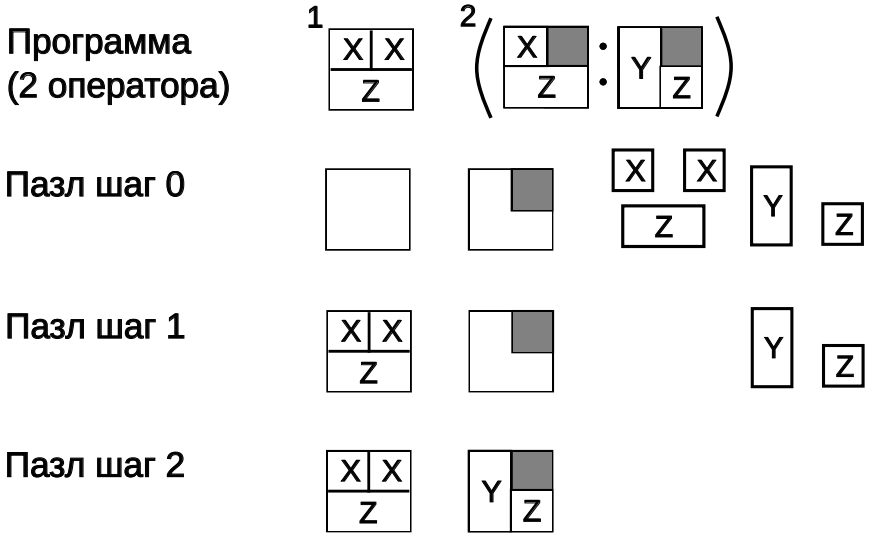


Рис. 24: Пример пошагового решения головоломки.

Процесс решения головоломки на Рис. 24 выглядит следующим образом:

1. Шаг 0 (Начальное состояние): Имеется головоломка с двумя незаполненными областями и набором доступных деталей.

2. Шаг 1 (Итерация по оператору 1): Решатель обрабатывает первый оператор. Он сканирует незаполненные области и находит соответствие с первой (полностью пустой) областью. Шаблон совпадает, и необходимые детали (две X и одна Z) имеются в наличии. Правило применяется: область заполняется, а соответствующие детали удаляются из набора.

3. Шаг 2 (Итерация по оператору 2): После того, как решатель посетит все области, пытаясь применить первый оператор, если всё ещё есть недостающие элементы, он переходит ко второму, условному оператору. Он находит соответствие по шаблону со второй, частично заполненной областью.

    - Первое правило условного оператора требует детали X и Z, которых нет в оставшемся наборе. Оно не применяется.
    - Решатель пробует второе правило, которое требует детали Y и Z. Эти детали есть в наличии. Правило успешно применяется, заполняя последнюю область.

Больше нет операторов для использования, но и свободных частей больше нет. Головоломка решена.

Алгоритм состоит из двух вложенных циклов: внешний цикл перебирает операторы в программе-решателе, а внутренний — незаполненные области головоломки. Поскольку количество операторов в программе является фиксированной константой, не зависящей от размера входной головоломки, общая сложность алгоритма линейно зависит от количества пустых областей.

Конечно, программа может не решить головоломку. Та же программа, которая использовалась ранее, не решит головоломку с Рис. 25. Шаблон в первом операторе соответствует пустой области, но элементы не совпадают. Поэтому программа зависает, решатель останавливается и сообщает, что решения нет.

На Рис. 26 представлена полная программа-решатель для головоломок типа 1, состоящая из 15 упорядоченных операторов. Поскольку каждая область состоит из четырех ячеек, существует $2^4 = 16$ возможных шаблонов, но одну из них, полностью закрашенную область, не обязательно учитывать.

Порядок операторов фиксирует стратегию: вначале обрабатываются наиболее жёстко ограниченные шаблоны (наибольшее число пред-

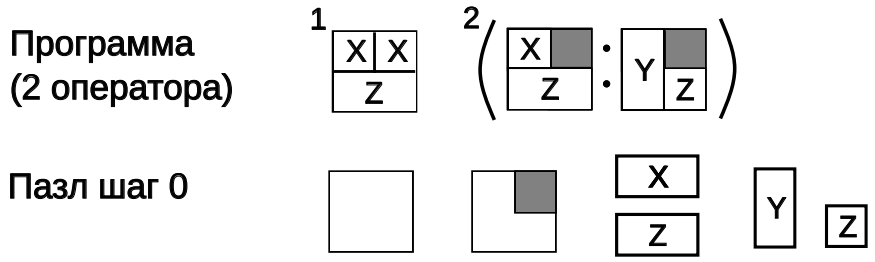


Рис. 25: Пример зависшей программы.

раскрашенных клеток и/или специфические их расположения, например диагональные пары), затем менее ограниченные.

Порядок операторов очень важен: его перестановка может приводить к тупикам, даже если решение существует.

Показателен пример на Рис. 27: если в утверждении 7 пытаться сначала использовать две маленькие детали X размера 1, то для оставшихся областей не останется подходящих комбинаций, и решатель зайдет в тупик. Однако, если сначала применить правило с одной крупной деталью X размера 2, головоломка успешно решается. Этот пример демонстрирует необходимость сначала использовать более крупные и менее гибкие детали, сохраняя более мелкие для последующих шагов.

2.3.8 Сложность

Согласно теореме, задача распределения регистров без сбросов (Spill-Free Register Allocation, SFRA) с предраскраской для элементарной программы P с K доступными регистрами решается за время $O(|P| \times K)$:

1. Для одной инструкции программы P решается одна головоломка с K областями (регистрами).

2. Применение одного правила к одной области занимает константное время O(1).

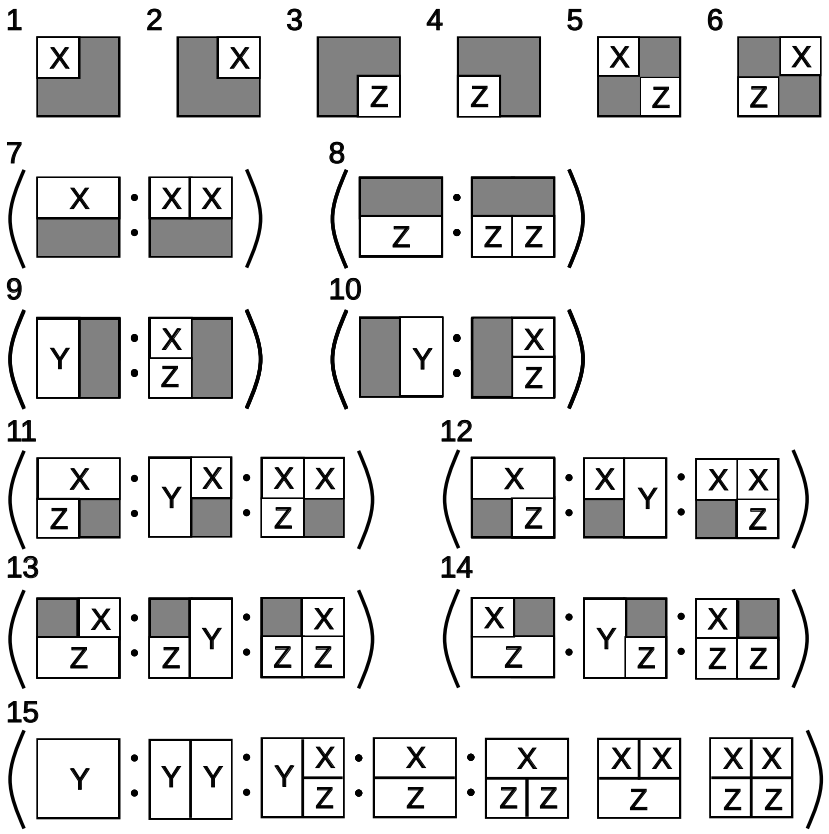


Рис. 26

3. Программа-решатель применяет к каждой области константное число правил.

4. Следовательно, решение головоломки для одной инструкции занимает время, пропорциональное количеству областей, то есть O(K).

5. Поскольку в программе |P| инструкций, общая сложность составляет O(|P| × K).

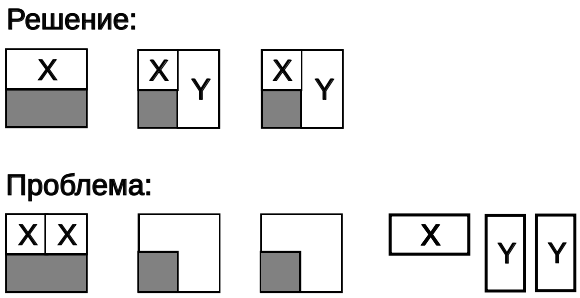


Рис. 27: Пример, демонстрирующий важность порядка правил: приоритет использования одной крупной детали (X размера 2) над двумя мелкими (X размера 1).

2.3.9 Spilling

Алгоритм обрабатывает программу инструкция за инструкцией. Если головоломка для текущей инструкции имеет решение, распределитель переходит к следующей. Однако, если головоломка нерешаема (например, из-за нехватки доступных регистров), запускается механизм сброса в память (spilling).

Процесс является итеративным:

1. Выбирается одна из переменных для выгрузки в память.
2. Соответствующая ей деталь удаляется из набора.
3. Предпринимается новая попытка решить головоломку.
4. Этот цикл повторяется до тех пор, пока не будет найдено решаемое состояние.

Теперь вопрос в том, какую деталь удалить. Ну, во-первых, мы не можем удалить детали X или Z, поскольку они представляют переменные, являющиеся непосредственными операндами текущей инструкции (используемыми или определяемыми). Поэтому мы должны удалить элемент Y.

Для выбора конкретной переменной из числа кандидатов используется эвристика Белади [6]: сбрасывается та переменная, следующее использование которой в программе находится дальше всего. После сброса переменная временно исключается из рассмотрения в последующих головоломках до тех пор, пока она снова не потребуется для выполнения какой-либо инструкции.

Выбор эвристического подхода здесь не случаен. Как гласит теорема, задача определения оптимального набора переменных для сброса является NP-полной. Поэтому вместо поиска вычислительно сложного, но идеального решения, применяется быстрая эвристика.

Теорема:

Выделение регистров в присутствии предраскрашивания (pre-coloring) и сброса (spilling) семейства переменных для элементарного представления — NP-полная задача.

2.3.10 Coalesing

Важной задачей при распределении регистров является объединение (coalescing) — стратегия, направленная на минимизацию лишних инструкций перемещения (copy, move). Ее суть заключается в том, чтобы одна и та же переменная программы по возможности занимала один и тот же физический регистр в смежных инструкциях. Если переменная перемещается из одного регистра в другой между двумя головоломками, в код необходимо вставить дополнительную операцию копирования для сохранения корректности программы.

Нахождение глобально оптимального распределения, которое минимизирует количество копий во всей программе, является NP-полной задачей. Поэтому в данном подходе, нацеленном на высокую скорость, применяется стратегия локального объединения.

Механизм работает следующим образом:

- Решение головоломки для текущей инструкции принимается с учетом распределения, полученного на предыдущем шаге, чтобы минимизировать количество перемещений между ними. Для этого обход программы выполняется в порядке, заданном деревом доминаторов.
- Сложность локального объединения зависит от наличия предопределенных регистров:
  - Если предопределенных регистров нет, задача решается за полиномиальное время.
  - При их наличии точный класс сложности неизвестен, и используется эвристическая стратегия best-effort.

Данный метод сознательно не стремится к глобальной оптимизации, так как основной целью является скорость распределения. Тем не менее, предложенный фреймворк не исключает использования более сложных, глобальных стратегий объединения в других реализациях.

2.3.11 Плюсы и минусы

Преимущества:

- Высокая производительность (для головоломок типов 0 и 1);
- Учитывает архитектурные особенности.

Недостатки:

- Неприменимость для некоторых архитектур (головоломки типа 2 не имеют общего решения);
- Локальный подход к решению (не учитывает глобальную картину распределения);

- Глобально оптимальные spilling/coalescing недостижимы (NP-трудно);
- Требует сложной предварительной подготовки кода (преобразование в элементарную форму).

Список литературы

[1] Poletto, Massimiliano and Sarkar, Vivek. Linear scan register allocation // ACM Transactions on Programming Languages and Systems. 1999. Vol. 21, No. 5. P. 895–913.

[2] Traub O., Holloway G., Smith M. D. Quality and speed in linear-scan register allocation // SIGPLAN Notices. 1998. Vol. 33, No. 5. P. 142–151.

[3] Schwarz K. CS143: Compilers. Lecture 17. Register Allocation. 2012. URL: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/17/Slides17.pdf (дата обращения: 2025).

[4] Quintão Pereira, Fernando Magno., Palsberg J. Register allocation by puzzle solving. Full version. 2008. URL: http://compilers.cs.ucla.edu/fernando/projects/puzzles/ (дата обращения: 2025).

[5] Quintão Pereira, Fernando Magno. Linear scan register allocation (a tutorial). URL: http://compilers.cs.ucla.edu/fernando/projects/puzzles/tutorial (дата обращения: 2025).

[6] L. Belady. A study of the replacement of algorithms of a virtual storage computer. IBM System Journal, 5:78–101, 1966.